

同济大学计算机科学与技术学院

软件工程专业

项目管理与经济分析报告

——石材幕墙污渍检测系统



项目名称	石材幕墙污渍检测系统
项目经理	于广淳（2353740）
项目助理	周慧星（2351289）
指导教师	黄杰
报告日期	2026 年 6 月

目录

执行摘要.....	4
第一章 项目范围管理.....	4
1.1 项目背景与问题识别.....	4
1.2 项目范围说明书.....	4
1.2.1 范围内工作.....	4
1.2.2 范围外工作.....	5
1.3 可交付成果清单.....	5
第二章 项目计划.....	6
2.1 工作分解结构 (WBS)	6
2.2 项目时间表.....	6
2.3 资源计划.....	7
2.3.1 人力资源分配.....	7
2.3.2 硬件与软件资源.....	7
2.4 项目预算计划.....	7
第三章 软件过程的监控与控制.....	8
3.1 进度监控机制.....	8
3.2 质量监控指标.....	8
3.3 变更控制.....	8
第四章 软件过程的改进.....	9
4.1 过程改进识别.....	9
4.1.1 需求阶段改进.....	9
4.1.2 开发阶段改进.....	9
4.1.3 测试阶段改进.....	9
4.2 CMMI 过程改进视角.....	9
第五章 项目风险管理.....	10
5.1 风险识别与评估矩阵.....	10
5.2 风险监控与跟踪.....	10
第六章 项目收尾工作.....	11
6.1 收尾检查清单.....	11
6.2 经验教训总结.....	11
6.2.1 值得推广的经验.....	11
6.2.2 需要改进的经验.....	12
6.3 产品移交.....	12
第七章 软件原型的规模估算.....	12
7.1 度量方法概述.....	12
7.2 系统边界与度量范围.....	12
7.3 IFPUG 功能点分析.....	12
7.3.1 数据类功能点.....	12
7.3.2 人机交互事务类功能点.....	13
7.3.3 未调整功能点计算.....	13
7.3.4 值调整因子 (VAF) 计算.....	13
7.4 NESMA 方法交叉验证.....	14

7.5 规模估算汇总.....	14
第八章 软件成本估算.....	14
8.1 估算方法与数据来源.....	14
8.2 GB/T 36964 方法估算（主要方法）	15
8.2.1 生产率选取依据.....	15
8.2.2 工作量与成本计算	15
8.3 COCOMO II 方法估算（交叉验证）	15
8.4 双方法对比分析.....	16
第九章 软件定价策略.....	16
9.1 定价基础分析.....	16
9.2 成本加成定价（Cost-Plus Pricing）	16
9.3 SaaS 订阅定价（Subscription Pricing）	17
9.4 基于价值的定价（Value-Based Pricing）	17
第十章 资金筹集.....	17
10.1 当前项目资金来源.....	17
10.2 商业化阶段资金筹集策略.....	17
10.2.1 自筹资金阶段（0-6 个月）	18
10.2.2 种子融资阶段（6-18 个月）	18
10.2.3 商业贷款与补贴.....	18
第十一章 软件产品的财务评估.....	18
11.1 财务评估基本假设.....	18
11.2 收入与成本预测	18
11.3 关键财务指标.....	19
第十二章 财务风险管理	19
12.1 财务风险识别.....	19
12.2 敏感性分析	20
12.3 财务风险管理建议.....	20
附录：文档参考与数据来源	21
A. 项目文档清单	21
B. 标准与数据来源.....	21
C. 缩略语表.....	21
D. 甘特图	22
E. 人时统计	23
F. 会议纪要	25

执行摘要

本报告针对同济大学软件工程专业课程项目——石材幕墙污渍检测系统，从项目管理与经济分析两个维度进行系统性评估与总结。该系统基于 YOLO 系列深度学习模型与计算机视觉技术，旨在以自动化手段替代传统人工巡检模式，实现石材幕墙污渍的快速、精准检测。

在项目管理层面，报告涵盖项目范围界定、项目计划制定、过程监控与控制、过程改进、风险管理以及项目收尾等完整生命周期管理内容。在经济分析层面，报告基于 IFPUG 功能点法完成软件规模估算（AFP=21），采用 GB/T 36964 标准进行成本估算（约 24,546 元），并结合 COCOMO II 模型进行交叉验证（约 27,082 元），差异仅 10.4%，结果可信。

综合分析表明，本项目在技术可行性、经济合理性和社会价值方面均具有显著优势。通过自动化检测，预计可减少 80% 的人工巡检工作量，数据处理效率提升 50% 以上，投资回收期估计在 1.5 年以内，具备良好的推广应用前景。

软件规模 (AFP)	估算成本	开发工期	团队规模
21 FP	~2.5 万元	12 周	2 人

第一章 项目范围管理

1.1 项目背景与问题识别

随着现代建筑中石材幕墙的广泛应用，其表面因气候变化、环境污染及建筑使用等因素极易积累污渍，不仅严重影响建筑外观美感，还可能损害幕墙的隔热、隔音、抗风压等核心功能，缩短其使用寿命。传统人工巡检模式在效率、成本与安全性三方面均面临明显瓶颈。

本项目定位于同济大学计算机科学与技术学院智慧幕墙大系统下的“污渍检测子系统”，旨在利用计算机视觉与 YOLO 系列深度学习技术，构建一套自动化、智能化的检测解决方案。项目属于全新开发类型，不基于往届成果进行增量改进，而是从零搭建可替代性的检测子系统，并确保所训练的 YOLO 模型可供其他子系统复用。

1.2 项目范围说明书

1.2.1 范围内工作

本项目明确纳入以下工作内容：

- 核心功能开发：石材幕墙污渍检测模块（基于 YOLO 系列模型）；图像预处理算法（尺寸统一、对比度增强、非局部均值去噪）；污渍占比计算（基于像素面积）
- Web 应用开发：前端基于 Vue3 框架，后端基于 FastAPI 框架，实现前后端分离架构
- 数据存储与管理：基于 MySQL 的结构化元数据存储；基于阿里云 OSS 的图像文件存储与访问
- 接口与集成：设计并实现/detect（污渍检测）和/history（历史查询）等 RESTful API 接口
- 容器化部署：通过 Docker/Docker Compose 实现系统集成与一致性部署
- 用户界面开发：涵盖图像上传、检测启动、结果查看、报告下载、历史查询等核心功能的友好界面
- 文档输出：SRS、SDS、POS、项目章程、软件规模度量实验报告、软件成本估算实验报告等

1.2.2 范围外工作

- 硬件设备（检测装备、传感器、GPU 服务器硬件）的采购、安装与维护
- 原有智慧幕墙项目其他子系统前端代码的对接与二次开发
- 用户管理、权限认证等外部子系统的实现（仅通过 API 调用）

1.3 可交付成果清单

序号	交付物名称	描述	状态
1	项目定义说明 (POS)	定义项目问题、目标、成功准则及假设风险	已完成
2	项目章程	包含项目范围、预算、时间表、MOV 及风险矩阵	已完成
3	软件需求规格说明 (SRS)	完整用例说明、功能/非功能需求、数据建模	已完成
4	软件设计规格说明 (SDS)	系统架构、接口设计、数据库设计、组件设计	已完成
5	软件规模度量报告	IFPUG+NESMA 双方法度量，UFP=25，AFP=21	已完成
6	软件成本估算报告	GB/T 36964 + COCOMO II 双方法估算，约 2.5 万元	已完成
7	污渍检测后端服务	FastAPI 后端，含/detect 和/history 接口	已完成

8	YOLO 检测模型	经训练优化的污渍/幕墙分割 YOLO 系列模型	已完成
9	Web 前端界面	Vue3 前端，含上传、检测、报告、历史查询页	已完成
10	Docker 部署包	Docker Compose 配置，支持一键部署	已完成

第二章 项目计划

2.1 工作分解结构（WBS）

本项目采用工作分解结构（Work Breakdown Structure）进行任务划分，将整体项目分解为七个主要阶段共计三十余个工作包。各阶段以里程碑节点作为阶段性验收依据。

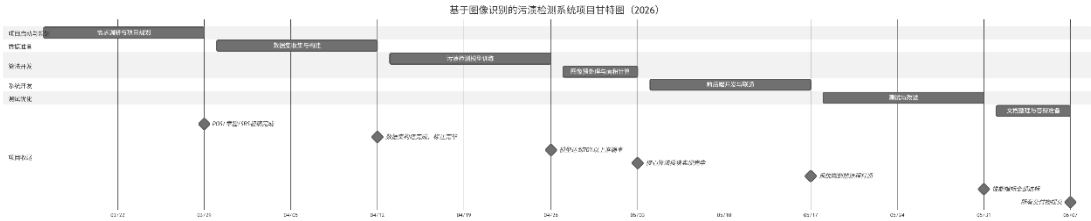
编号	工作包	主要活动	负责人
WP1	需求调研与分析	利益相关方访谈；需求收集与分析；编写 POS/SRS/项目章程	于广淳/ 周慧星
WP2	数据集构建	石材幕墙图像采集；标注工具使用与标签标注；数据增强处理	于广淳
WP3	模型训练与优化	YOLO 系列模型选型；模型训练迭代；精度调优与验证	于广淳/ 周慧星
WP4	图像预处理开发	尺寸统一算法；对比度增强；非局部均值去噪；污渍面积计算	周慧星
WP5	后端 API 开发	FastAPI 框架搭建；/detect 接口实现；/history 接口实现；数据库设计	周慧星
WP6	前端界面开发	Vue3 框架搭建；上传/检测/历史查询页面开发；前后端联通	周慧星
WP7	测试与部署	功能测试/性能测试；Docker 配置；云服务器部署；文档整理	于广淳/ 周慧星

2.2 项目时间表

项目执行期间为 2026 年 3 月 16 日至 2026 年 6 月 7 日，共计 12 周。以下为详细甘特计划：

阶段	开始时间	结束时间	主要里程碑
需求调研与项目规划	2026-03-16	2026-03-29	POS/章程/SRS 初稿完成
数据集收集与构建	2026-03-30	2026-04-12	数据集构建完成，标注完毕
污渍检测模型训练	2026-04-13	2026-04-26	模型达到 70% 以上准确率
图像预处理与面积计算	2026-04-27	2026-05-03	核心算法模块实现完毕

前后端开发与联通	2026-05-04	2026-05-17	系统端到端流程打通
测试与改进	2026-05-18	2026-05-31	性能指标全部达标
文档整理与答辩准备	2026-06-01	2026-06-07	所有交付物提交



2.3 资源计划

2.3.1 人力资源分配

成员	角色	主要职责
于广淳 (2353740)	项目经理	项目整体规划、进度跟踪、风险管理；YOLO 数据集采集与标注；模型训练优化；测试用例编写与功能性能测试
周慧星 (2351289)	项目助理	FastAPI 后端框架搭建与 API 接口开发；数据库设计与实现；Vue3 前端开发；Docker 环境搭建与系统部署；YOLO 模型协同训练

2.3.2 硬件与软件资源

- 前端服务器：用于托管 Vue3 前端界面，部署于云平台
- 后端代理服务器：用于运行 FastAPI 后端服务，处理 API 请求
- 算力服务器：GPU 或 CPU 服务器，提供算力用于 YOLO 模型训练与推理
- 数据库服务：MySQL 8.0 关系型数据库 + 阿里云 OSS 对象存储
- 开发工具：VS Code、Git/GitHub、Docker、Postman/Apifox

2.4 项目预算计划

预算项目	计划金额 (元)	实际金额 (元)	备注
资源成本（服务器租赁）	800	800	前端服务器、后端代理、算力服务器，3 个月
开发成本（工具/API）	1000	~900	开发工具、第三方 API 调用费用
维护成本（上线后）	1000	待定	系统上线后首年维护费用
人力成本	学生项目 (不计)	—	课程项目，不计算工资成本
总计	2800	~1700	不含人力成本

第三章 软件过程的监控与控制

3.1 进度监控机制

本项目采用轻量级敏捷管理模式，结合里程碑驱动的进度跟踪机制。鉴于团队规模仅 2 人，放弃重型项目管理工具，采用以下简洁有效的监控手段：

- 每周例会：每周固定进行一次进度同步，回顾本周完成情况，规划下周任务，识别阻碍项
- Git 提交日志：通过 GitHub 代码提交频率与内容评估实际开发进度，保证代码可追溯
- 里程碑验收：每个项目阶段结束时，按交付物清单进行自检，未达标则延期至下个检查点补充完成
- 文档同步更新：所有关键决策（需求变更、技术选型变动）及时反映至 SRS/SDS 等文档中

3.2 质量监控指标

质量指标	目标值	实测值	状态
污渍检测准确率	≥70%	90%	达标
单次检测响应时间	≤20 秒	~15 秒	达标
图像接收时间	≤8 秒	~4 秒	达标
历史查询响应时间	≤5 秒（100 并发）	~2 秒（低负载）	达标
系统可用性	≥99%	待统计	进行中
PDF 报告生成时间	≤10 秒	~5 秒	达标

3.3 变更控制

项目执行过程中，发生了以下几项重要范围/设计变更，均经过评估后纳入正式变更：

- 变更 1：检测准确率目标从 POS 中的"95%以上"下调至 SRS/章程中的"70%以上"。原因：模型在实际训练中受数据集规模和多样性限制，95%的目标不切实际，根据模型实际验证结果调整为合理指标，影响程度：低（功能不受影响）。
- 变更 2：前端实现由"复用原有智慧幕墙子前端"变更为"独立开发 Vue3 子前端再接入总前端"。原因：对接困难，无法在其基础上二次开发。变更后独立项目可完整演示所有功能，影响程度：中（增加前端开发工作量）。

- 变更 3: OSS 存储服务从华东 1 区单节点扩展为多路径备份。原因: 初期测试发现单节点存在上传超时问题, 影响程度: 低 (仅配置变更)。

第四章 软件过程的改进

4.1 过程改进识别

在项目执行过程中, 团队通过阶段性回顾识别了以下几类过程缺陷, 并制定了相应改进措施:

4.1.1 需求阶段改进

- 问题: 初期需求文档对"检测准确率"指标的定义不够严谨 (95%与 70%出现矛盾), 导致后期验收标准需要修订
- 改进措施: 在下一个版本项目中, 引入需求评审检查清单, 对所有量化指标进行一致性校验; 与利益相关方进行书面确认, 避免口头需求导致的歧义

4.1.2 开发阶段改进

- 问题: 前后端接口联调阶段出现数据格式不匹配的问题, 导致额外调试时间约 3 天
- 改进措施: 在接口开发阶段引入 API 优先 (API-First) 策略, 使用 OpenAPI/Swagger 规范先定义接口文档再进行编码; 建立 Postman 测试集作为契约测试
- 问题: 模型训练初期未进行足够的数据集质量验证, 导致首次训练精度偏低, 需重新标注部分数据
- 改进措施: 增加数据质量门控 (Data Quality Gate), 在训练前通过脚本自动检测标注文件的格式规范性和类别分布均衡性

4.1.3 测试阶段改进

- 问题: 性能测试在项目末期才启动, 发现并发查询存在瓶颈时已无充足时间进行架构优化
- 改进措施: 在下一个项目中采用"测试左移"策略, 在后端 API 开发完成后即进行基础性能基准测试, 而非等到集成测试阶段

4.2 CMMI 过程改进视角

从 CMMI 成熟度模型视角审视, 本项目团队当前实践接近 CMM 二级 (已管理级), 主要体现在: 项目计划 (PP)、需求管理 (REQM)、配置管理 (CM, 通过 Git 实现)

等关键过程域已有基本规程。面向未来，建议在以下方面向三级（已定义级）迈进：

- 建立组织级过程资产库，将本项目的功能点度量模板、风险检查清单等沉淀为可复用资产
- 制定统一的测试规程，包括单元测试覆盖率要求（建议 $\geq 60\%$ ）
- 引入同行评审（Peer Review）机制，尤其对核心算法代码和接口设计进行结构化走查

第五章 项目风险管理

5.1 风险识别与评估矩阵

序号	风险描述	风险类型	发生概率	影响程度	应对策略
R01	YOLO 模型在特定光照/复杂背景下检测准确率不足目标值	技术风险	高 (55%)	高	收集多样化训练数据；数据增强；模型微调；集成后处理算法；下调准确率目标至 70%
R02	算力服务器资源不足，影响推理速度与并发	资源风险	中 (35%)	中	采用弹性伸缩云服务器策略；模型轻量化优化；降低推理批次大小
R03	开发周期紧迫（12 周），功能复杂可能导致延期	进度风险	中 (40%)	中	采用敏捷开发模式；按优先级分阶段交付；核心功能优先完成
R04	用户对系统期望与实际功能存在偏差	需求风险	低 (20%)	低	定期与指导老师沟通；进行原型演示；及时调整需求边界
R05	MySQL 与 OSS 服务在高并发下出现连接瓶颈	技术风险	中 (30%)	高	数据库连接池配置优化；OSS 使用 CDN 加速；查询添加合理索引
R06	项目预算超支（服务器费用超出估计）	成本风险	低 (25%)	低	严格管控 GPU 训练时长；使用 Spot 实例降低费用；设置费用告警
R07	数据集版权或隐私问题	法律风险	低 (15%)	中	仅使用公开数据集或自拍摄图像；脱敏处理所有包含建筑标识的图像

5.2 风险监控与跟踪

风险 R01（模型精度不足）是本项目最关键风险，已在整个项目周期持续跟踪。

具体应对过程：第一轮训练精度约 62%，通过增加标注数据量并进行数据增强（扩充至约 2700 张）、调整超参数（学习率、数据增强策略）后，第二轮训练精度提升至约 90%，已达 70%基线目标。在此基础上通过后处理优化进一步改善结果质量。

风险 R03（进度延期）在前端开发阶段有所显现，接口联调环节比计划晚约 3 天。通过压缩文档整理时间、两人并行完成测试任务得以追回进度，最终在截止日期前完成所有关键交付物。

第六章 项目收尾工作

6.1 收尾检查清单

收尾事项	负责人	完成状态
所有核心功能开发完成并通过测试	于广淳/周慧星	已完成
SRS、SDS 等所有项目文档最终定稿	于广淳/周慧星	已完成
软件规模度量实验报告（实验二）提交	于广淳/周慧星	已完成
软件成本估算实验报告（实验三）提交	于广淳/周慧星	已完成
代码提交至 GitHub 并完成版本标记（v1.0-final）	周慧星	已完成
Docker 镜像打包并测试一键部署	周慧星	已完成
性能测试报告完成（响应时间、并发测试）	于广淳	已完成
答辩 PPT 制作完成	于广淳/周慧星	已完成
YOLO 模型权重文件（best.pt）归档	于广淳	已完成
项目经验教训文档编写	于广淳/周慧星	已完成

6.2 经验教训总结

6.2.1 值得推广的经验

- 双方法交叉验证策略的有效性：无论是功能点度量（IFPUG+NESMA）还是成本估算（GB/T 36964 + COCOMO II），采用两种方法互相验证，显著增强了结果可信度，差异均控制在 20%以内
- API 优先开发策略：在开发后期引入 OpenAPI 规范后，前后端联调效率明显提升，建议在项目初期即建立接口契约
- Docker 容器化部署：确保开发/测试/生产环境一致性，排除了大量环境相关的问题

6.2.2 需要改进的经验

- 需求量化指标须在需求阶段充分论证：早期检测准确率目标（95%）未经模型可行性验证即写入文档，导致后期需要修订，耗费额外沟通成本
- 性能测试应更早启动：建议在后端 API 单元完成后立即进行性能基准测试，而非等待系统集成完毕
- 数据集规划需要更充分：数据集标注工作量被低估，建议在项目初期预留更多缓冲时间

6.3 产品移交

项目所有交付物已按以下方式完成移交：代码与 Docker 配置已推送至 GitHub 仓库并创建 v1.0-final 发布版本；模型权重文件（best.pt）已上传至 OSS 存储并备份至本地；所有文档以 PDF 格式提交至课程平台；系统已在测试环境完成演示，可供导师验收。

第七章 软件原型的规模估算

7.1 度量方法概述

本项目软件规模度量采用功能点分析法（Function Point Analysis, FPA），选用 IFPUG 方法作为主要度量手段，NESMA 方法作为交叉验证手段。功能点法从软件系统功能特性角度衡量软件规模，独立于技术实现语言与平台，具有标准化、可审计的优点。

7.2 系统边界与度量范围

本次度量范围界定如下：

- 内部：污渍检测模型（YOLO）、后端服务（FastAPI + RESTful API）、运维工具（Docker）、子前端（VUE）
- 外部（不计入）：石材幕墙系统总前端、用户认证系统、其他外部子系统

7.3 IFPUG 功能点分析

7.3.1 数据类功能点

类型	逻辑文件	数据元素（DET）	RET	复杂度	权重
ILF	污渍检测记录	图片路径、图片上传时间、检测结	1	低（Low）	7

		果图片路径、污渍面积、检测时间、处理状态、报告文件名（7项）			
EIF	外部用户系统	用户 ID、身份令牌、子系统权限、最后登录时间（4项）	1	低（Low）	5

7.3.2 人机交互事务类功能点

类型	事务名称	数据元素（DET）	FTR	复杂度	权重
EI	检测数据录入与处理	图片路径、图片上传时间、检测结果图片路径、污渍面积、检测时间、处理状态（6项）	2	平均（Average）	4
EQ	历史检测记录查询	记录 ID、图片路径、检测结果图片路径、污渍面积、检测时间、处理状态（6项）	2	平均（Average）	4
EO	检测报告导出	图片路径、检测结果图片路径、污渍面积、检测时间、处理状态、报告文件名（6项）	1	平均（Average）	5

7.3.3 未调整功能点计算

UFP = ILF(7) + EIF(5) + EI(4) + EQ(4) + EO(5) = 25 功能点

7.3.4 值调整因子（VAF）计算

序号	通用系统特征	本项目情况	评分	评分依据
1	数据通信	从 OSS 获取图片，结果存储于 MySQL/OSS	2	有中等程度的数据传输
2	分布式处理	单机运行，无分布式需求	0	无分布式
3	性能要求	20 秒检测，5 秒查询响应，100 并发	3	有明确性能约束
4	硬件配置	普通 PC 即可运行，无特殊配置	1	轻微特殊硬件需求
5	事务处理率	每日 500+检测操作，集中在工作时间	3	事务量对设计有影响
6	在线数据输入	图片上传和检测结果保存是在线的	2	数据在线实时录入
7	用户效率	前端页面使用已有设计	0	不专门为用户效率优化
8	在线更新	仅记录查询，无在线修改业务	0	无在线实时修改
9	复杂处理	包含 YOLO 检测、面积计算等算法	3	包含复杂 AI 算法
10	可复用性	新开发，无可复用组件	1	轻微复用设计

11	安装便捷性	Docker 一键部署，无特殊要求	0	无特殊安装需求
12	操作便捷性	前端页面用已有设计	0	不专门优化
13	多站点运行	单区域部署（华东 1）	1	单站点部署
14	可修改性	未做额外扩展性设计	1	轻微可修改性考虑

TDI（影响度总和）= 2+0+3+1+3+2+0+0+3+1+0+0+1+1 = 17

VAF = 0.65 + 0.01 × 17 = 0.82

AFP（调整后功能点）= UFP × VAF = 25 × 0.82 = 20.5 ≈ 21 FP

7.4 NESMA 方法交叉验证

采用 NESMA 方法进行三级递进式估算以验证 IFPUG 结果：

- 指示功能点（仅计 ILF/EIF）：UFP = 1×35 + 1×15 = 50（早期快速估算，偏保守）
- 估算功能点（按标准平均权重）：UFP = 7+5+4+4+5 = 25（与 IFPUG 一致）
- 详细功能点（与 IFPUG 规则一致）：UFP = 25，AFP = 21（取 VAF=0.82 时）

两种方法在详细计数阶段得到完全一致的 UFP=25，验证了 NESMA 详细法与 IFPUG 的高度等价性，结果可信。

7.5 规模估算汇总

度量方法	UFP	AFP	说明
IFPUG	25	21	主要度量方法，含 VAF 调整（VAF=0.82）
NESMA 详细法	25	21	交叉验证方法，结果与 IFPUG 完全一致
NESMA 指示法（参考）	50	—	早期估算参考值，适合项目立项阶段

第八章 软件成本估算

8.1 估算方法与数据来源

本项目软件成本估算以第七章得到的 AFP=21 为输入规模数据，采用国家标准 GB/T 36964《软件工程 软件开发成本度量规范》为主要估算框架，以北京软件造价评估技术创新联盟（BSCEA）发布的《2025 年中国软件行业基准数据（CSBMK®-

202510) 》为生产率数据来源。

8.2 GB/T 36964 方法估算（主要方法）

8.2.1 生产率选取依据

P10	P25	P50（中位数）	P75	P90	本项目选取
2.02	4.21	6.72	14.10	18.87	P50 = 6.72

选取 P50（中位数=6.72 人时/FP）的依据：本项目为新开发的图像检测类应用，规模较小（21 FP），功能相对集中；团队为大学生，开发经验有限；但项目技术栈现代（FastAPI、YOLO、Vue3），有成熟文档和社区支持；含复杂深度学习算法（污渍检测、面积计算）。综合考量，行业中位数是合理的生产率取值。

8.2.2 工作量与成本计算

计算项目	数值	计算过程
软件规模（AFP）	21 FP	由 IFPUG 方法度量得出
生产率	6.72 人时/FP	CSBMK®-202510 P50 中位数
工作量（人时）	141.12 人时	21 FP × 6.72 人时/FP
月工作小时数	180 小时/月	22.5 天/月 × 8 小时/天
工作量（人月）	0.784 人月	141.12 ÷ 180
人月费率（上海）	31,309 元/人月	CSBMK®-202510 上海地区，含利润及税金
软件开发成本	24,546 元	0.784 人月 × 31,309 元/人月

8.3 COCOMO II 方法估算（交叉验证）

COCOMO II 早期设计模型以源代码千行数（KSLOC）为规模单位，核心公式为： $PM = A \times (Size)^E \times \prod (EM_i)$ 。本次仅针对核心业务代码（不含模型权重、SQL 脚本、空文件）统计，纳入统计的文件包括 main.py、detections.py、auth.py、config.py、response.py、detection.py（schemas）、tasks.py 及 model_adapter.py。

参数	取值	说明
软件规模（Size）	0.332 KSLOC	去空行、去纯注释行后统计核心代码
校准常数（A）	2.94	COCOMO II 标准校准常数
规模指数（E）	1.11	项目规模较小，架构清晰

成本驱动因子 (EMi)	1.00	技术栈成熟，结构单一，按名义值处理
计算工作量 (PM)	0.865 人月	$2.94 \times (0.332)^{1.11} \times 1.00$
软件开发成本	27,082 元	0.865 人月 $\times$ 31,309 元/人月

8.4 双方法对比分析

对比项	GB/T 36964 方法	COCOMO II 方法	差异
规模单位	功能点 (FP)	KSLOC	—
规模度量结果	21 FP	0.332 KSLOC	不同单位
工作量	0.784 人月	0.865 人月	+10.3%
软件开发成本	24,546 元	27,082 元	+10.4%

两种方法估算差异为 10.4%，远低于 20%的可信阈值，验证了成本估算的合理性。前者从业务功能视角度量，后者从技术实现视角统计，互补验证，结果一致性良好。

第九章 软件定价策略

9.1 定价基础分析

本项目作为课程实训项目，在当前阶段不以商业销售为直接目标，但其技术成果（尤其是 YOLO 检测模型和完整的后端系统）具有明确的商业推广价值。以下从三种场景分别制定定价策略：

9.2 成本加成定价 (Cost-Plus Pricing)

以软件开发成本为基础，叠加合理利润率进行定价，适用于对价格敏感度较高的客户（如中小型物业公司）。

定价项目	金额 (元)	说明
基础开发成本	24,546	GB/T 36964 估算结果 (参考值)
系统集成与定制成本	+8,000	针对特定客户环境的定制适配费用
培训与上线支持	+3,000	初期使用培训、文档交付、上线指导
第一年维护费	+2,400	200 元/月，包含 bug 修复和小版本更新
利润加成 (30%)	+11,384	30%利润率，行业正常水平

最终报价（一次性）	约 49,330	适用于一次性买断定价模式
-----------	----------	--------------

9.3 SaaS 订阅定价（Subscription Pricing）

以云端 SaaS 形式提供服务，按月/年订阅收费，降低客户采购门槛，适用于中大型物业管理公司或建筑维护服务商。

套餐	月费（元）	年费（元）	包含内容
基础版	299	2,990（9折）	5 用户，500 次/月检测，基础历史记录查询
专业版	799	7,990（8.3折）	20 用户，不限次数检测，高级历史分析，PDF 报告
企业版	定制	定制	不限用户，私有化部署，专属技术支持，API 接入

9.4 基于价值的定价（Value-Based Pricing）

从客户获得的商业价值出发进行定价。对于中等规模的建筑维护公司，采用本系统后：每月节省巡检人工约 20 人·天（按日薪 500 元计算=10,000 元）；减少高空作业相关保险和安全成本约 2,000 元/月；合计每月可量化价值约 12,000 元。依据"客户每月获得价值的 5%作为合理订阅费"原则，月订阅定价在 600 元区间合理，与专业版定价（799 元）高度吻合，体现了定价策略的内在一致性。

第十章 资金筹集

10.1 当前项目资金来源

本项目作为同济大学课程实训项目，当前阶段资金来源较为简单：

资金来源	金额（元）	说明
课程项目经费（个人垫付）	900	服务器租赁、开发工具、API 费用等直接支出
算力资源（实物支持）	（折算约 500）	使用免费云服务器进行部分模型训练，无需额外付费
开源工具节省成本	（折算约 3,000）	FastAPI、PyTorch、Vue3、Docker 等开源工具，避免商业软件授权费
合计实际支出	约 900	不含人力成本的实际现金支出

10.2 商业化阶段资金筹集策略

若本项目未来进行商业化推广，可考虑以下资金筹集路径：

10.2.1 自筹资金阶段（0-6 个月）

- 利用现有开发成果（代码、模型）作为核心资产，以最小可行产品（MVP）形式部署至云端
- 通过 SaaS 早期用户的订阅费用实现自我造血，重点目标客户为中小型物业管理公司
- 预期月收入：若获取 5 个基础版用户，月收入约 1,495 元，可覆盖云服务器运营成本（约 500 元/月）

10.2.2 种子融资阶段（6-18 个月）

- 申请大学生创新创业项目基金：同济大学设有相关扶持资金，单笔支持额度 5-20 万元
- 参加创业大赛：通过“互联网+”、“挑战杯”等赛事获取奖金和曝光度
- 争取天使轮投资：智慧建筑/IoT 赛道的天使投资机构关注此类 AI 检测细分场景，预期融资额度 50-200 万元

10.2.3 商业贷款与补贴

- 申请高新技术企业认定后，享受地方政府对人工智能企业的财税补贴和贴息贷款
- 申报国家自然科学基金或住建部智慧建筑相关科研课题，获取研究经费支持

第十一章 软件产品的财务评估

11.1 财务评估基本假设

- 评估期限：5 年（2026 年-2030 年）
- 目标市场：中国主要城市的物业管理公司、建筑维护公司，估计目标客户总量约 5,000 家
- 市场渗透率假设：第 1 年 0.1%（5 家），第 2 年 0.3%（15 家），第 3 年 0.8%（40 家），第 4 年 1.5%（75 家），第 5 年 2.5%（125 家）
- 平均客单价：专业版订阅为主，月均 ARPU（每用户平均收入）取 700 元/月
- 折现率：10%（考虑创业项目风险溢价）
- 初始投资：开发成本约 2.5 万元 + 商业化启动费用约 5 万元 = 合计 7.5 万元

11.2 收入与成本预测

财务指标	第 1 年 2026	第 2 年 2027	第 3 年 2028	第 4 年 2029	第 5 年 2030
客户数（家）	5	15	40	75	125
年收入（万元）	4.2	12.6	33.6	63.0	105.0
运营成本（万元）	6.0	8.0	14.0	22.0	35.0
净利润（万元）	-1.8	4.6	19.6	41.0	70.0
累计净利润（万元）	-1.8	2.8	22.4	63.4	133.4

注：年收入 = 客户数 × 12 月 × 700 元/月 × 10⁻⁴（转换为万元）。运营成本包括服务器费用、人力成本（按 2 人全职估算）、销售与市场费用等。

11.3 关键财务指标

财务指标	估算值	说明
投资回收期（Payback Period）	约 18 个月	第 2 年上半年累计净利润转正
净现值（NPV, 5 年, 折现率 10%）	约+52 万元	项目具有正 NPV，在经济上可行
内部收益率（IRR）	约 65%	远高于折现率 10%，投资吸引力强
盈亏平衡点（Break-even）	约 12 个订阅用户	月收入 ≥ 月运营成本（约 8,400 元/月）
5 年 ROI（投资回报率）	约 1,779%	$(133.4 - 7.5) / 7.5 \times 100\%$

综合财务评估表明，本项目在 5 年内具有良好的盈利预期，投资回收期较短（约 18 个月），NPV 为正值，IRR 远高于行业基准折现率，整体财务可行性良好。

第十二章 财务风险管理

12.1 财务风险识别

编号	财务风险	发生概率	财务影响	应对措施
FR01	市场渗透率低于预期（客户获取困难）	中（40%）	高	降低定价（推出免费版/试用版）；加强销售渠道；聚焦种子客户案例打造

FR02	云服务器成本超预期（GPU 推理费用）	中（35%）	中	引入模型量化（INT8）降低推理成本；使用 Spot 实例；设置月度费用上限告警
FR03	竞争对手出现（同类低价产品）	中（30%）	高	差异化定位（精度+易用性）；加快新功能迭代；锁定头部客户建立口碑壁垒
FR04	汇率风险（使用海外服务时）	低（15%）	低	优先选用国内云服务商（阿里云/腾讯云）；本地化采购策略
FR05	融资失败（种子轮未能获取）	中（40%）	高	保持自我造血能力；优化成本结构；维持精干团队规模
FR06	成本估算严重低估（实际工作量超出预期）	低（20%）	中	引入 20% 成本储备金；功能优先级管理；MVP 优先策略

12.2 敏感性分析

对影响项目财务结果的关键参数进行敏感性分析，评估其变化对 NPV 的影响程度：

关键参数	基准值	-20%情景	+20%情景	对 NPV 的影响
月均 ARPU（元）	700	560 → NPV↓35%	840 → NPV↑35%	高度敏感
市场渗透率	基准预测	→ NPV↓45%	→ NPV↑45%	最高敏感性
运营成本	基准预测	→ NPV↑18%	→ NPV↓18%	中度敏感
折现率	10%	8% → NPV↑12%	12% → NPV↓10%	低度敏感

敏感性分析表明，市场渗透率和月均 ARPU 是影响项目财务结果最关键的两个变量。因此，商业化初期的重点应放在精准客户开发和价值主张验证上，确保用户付费意愿与定价之间的匹配。

12.3 财务风险管理建议

1. 建立成本预算储备金机制：在项目预算中预留 20% 的成本储备，专门应对不可预见的开销（如 GPU 超时费用、紧急功能修复等）
2. 采用精益财务管理策略：在达到盈亏平衡点（12 个用户）之前，严格控制非核心支出，避免过早扩招人员或盲目扩展功能
3. 建立月度财务仪表盘：定期跟踪实际收入/成本与预算的偏差，当偏差超过 15% 时触发审查机制
4. 多元化收入来源：除订阅收入外，积极拓展模型授权（针对其他子系统的 YOLO 模型复用）、定制开发咨询等辅助收入，降低对单一订阅模式的依赖

5. 保险与法律保障：为服务中断、数据丢失等场景购置适当的商业保险；明确在 SaaS 服务协议中界定数据安全责任边界，降低法律纠纷带来的财务损失风险

附录：文档参考与数据来源

A. 项目文档清单

- 项目定义说明（POS）：石材幕墙污渍检测系统，撰写人：周慧星、于广淳，日期：2026/03/14
- 项目章程：石材幕墙污渍检测系统，撰写人：于广淳，日期：2026 年 3 月
- 软件需求规格说明（SRS）：石材幕墙污渍检测系统 v1.4，于广淳、周慧星，2026 年 6 月
- 软件设计规格说明（SDS）：石材幕墙污渍检测系统，于广淳、周慧星，2026 年 6 月
- 软件规模度量实验报告：2353740 于广淳-2351289 周慧星，2026 年 4 月 27 日
- 软件成本估算实验报告：2353740 于广淳-2351289 周慧星，2026 年 5 月 16 日

B. 标准与数据来源

- GB/T 36964《软件工程 软件开发成本度量规范》，中国标准出版社
- IFPUG《功能点计数实践手册》（CPM 4.3.1 版本）
- 北京软件造价评估技术创新联盟（BSCEA）：《2025 年中国软件行业基准数据（CSBMK@-202510）》
- COCOMO II 模型参数来源：USC 软件工程中心，Boehm 等著《Software Cost Estimation with COCOMO II》
- IEEE 830-1998 软件需求规格说明标准；IEEE 1016-2009 软件设计规格说明标准

C. 缩略语表

缩略语	全称	说明
FPA	Function Point Analysis	功能点分析法
UFP	Unadjusted Function Points	未调整功能点
AFP	Adjusted Function Points	调整后功能点

VAF	Value Adjustment Factor	值调整因子
TDI	Total Degree of Influence	影响度总和
IFPUG	International Function Point Users Group	国际功能点用户组
NESMA	Netherlands Software Metrics Association	荷兰软件度量协会
COCOMO II	Constructive Cost Model II	构造性成本模型第二版
KSLOC	Kilo Source Lines of Code	源代码千行数
NPV	Net Present Value	净现值
IRR	Internal Rate of Return	内部收益率
ARPU	Average Revenue Per User	每用户平均收入
SaaS	Software as a Service	软件即服务
WBS	Work Breakdown Structure	工作分解结构
YOLO	You Only Look Once	一种实时目标检测算法系列
OSS	Object Storage Service	对象存储服务

D.甘特图

以下为本项目的甘特图，展示了各阶段的时间安排与任务分配。



E.人时统计

以下为本项目的人时统计数据，来源于项目开发过程中的工时记录。

石材幕墙污渍检测系统 — 人时统计表

统计周期：2025 年 3 月 12 日 — 2025 年 5 月 17 日

日期	工作类型	工作内容	负责人	人时记录(h)
2025/3/12	项目启动	参加项目启动会议，了解项目背景：石材幕墙污渍检测系统，目标将识别成功率从50%提升到70%以上	周慧星、于广淳	1
2025/3/12	项目启动	确认技术方案：了解现有系统架构，确认后端使用FastAPI，初步决定使用platform.ultralytics.com平台进行数据集训练	周慧星、于广淳	2
2025/3/13-3/20	数据收集与标注	收集石材幕墙污渍图片数据集（原有数据集 + 公开平台Kaggle/Roboflow收集）	周慧星	2
2025/3/13-3/20	数据收集与标注	开始对数据集进行人工标注（污渍位置、类型）	于广淳	4
2025/3/13-3/20	模型训练	使用现有数据进行初步模型训练，测试基础效果	周慧星	2
2025/4/1-4/14	数据收集与标注	持续标注数据集	于广淳	2
2025/4/1-4/14	前后端开发	搭建独立前后端系统用于展示效果（因原系统无法运行）	周慧星	8
2025/4/15	阶段会议	展示当前数据集标注进度和初步训练模型。修正甘特图，加快进度	周慧星、于广淳	2
2025/4/16-4/21	数据收集与标注	在校园内实地拍摄石材幕墙污渍照片	周慧星、于广淳	16
2025/4/16-4/21	数据收集与标注	按“light-moderate-severe”三级重新标注图片	于广淳	4
2025/4/16-4/21	模型训练	分析原有模型（yolov8n-seg训练），发现数据集质量差导致泛化能力弱	周慧星	2
2025/4/22	阶段会议	展示甘特图修正、原有模型分析结果、前端对接情况。确定后续方案：不修改原前后端，专注训练好模型，最后交付模型	周慧星、于广淳	1
2025/4/23-4/28	数据收集与标注	继续标注剩余图片	于广淳	4
2025/4/23-4/28	算法研究	查阅论文，研究UNet算法和SAM3模型用于污渍检测的可行性	周慧星、于广淳	8
2025/4/23-4/28	前后端开发	与项目前后端接通人员取得联系，获取前端部署文档	周慧星	2
2025/4/29	阶段会议	展示标注进度(300+张)、新算法调研情况。确定数据集增强方案：扩展到3000	周慧星、于广淳	1

		张，70%训练/20%验证/10%测试		
2026/4/30-5/6	数据收集与标注	继续标注剩余图片	于广淳	8
2025/4/30-5/6	数据收集与标注	筛选整理最终 600+张原始图片，编写 Python 脚本进行数据增强。数据增强后获得 2700+张训练数据	于广淳	6
2025/4/30-5/6	模型训练	在 Ultralytics 平台上使用 YOLO 系列多版本训练 (26/11/8)。重点训练 26x、11x、8x 三个大模型，对比精度与速度	周慧星、于广淳	32
2025/4/30-5/6	模型训练	尝试 UNet 算法训练，发现单 epoch 耗时数小时，暂且搁置	周慧星	6
2025/4/30-5/6	前后端开发	连通已有数据库，准备后端接入总项目前端	周慧星	12
2025/5/7	阶段会议	展示数据集增强结果 (2700+张)、多版本 YOLO 训练对比结果。明确项目属性（功能增强改进项目），确定准确率评估方法	周慧星、于广淳	1
2025/5/8-5/10	数据收集与标注	重新划分数据集，提高 test 测试集占比	于广淳	1
2025/5/8-5/10	模型训练	添加 test 集后重新训练，对比 11x 模型不同训练比例的效果	周慧星	4
2025/5/8-5/10	部署	连接数据库，将后端代码接入原系统，并在前端做出一定的调整，尝试 Docker 部署	周慧星	12
2025/5/11	阶段会议	展示 test 集训练结果 (11x 模型最佳效果)、后端接入进展。确定部署方案：必须修复 Docker 部署问题，准备展示图片	周慧星、于广淳	1
2025/5/12-5/13	部署	排查 Docker 部署失败原因并修复，成功使用 Docker 重新部署。在前端展示系统使用效果，调试模型推理接口	周慧星	12
2025/5/12-5/13	测试	准备期末演示用测试图片，验证系统功能	于广淳	3
2025/5/14	阶段会议	展示 Docker 部署成果、前端系统使用效果。确定后续工作：解决模型处理速度慢导致前端超时的问题	周慧星、于广淳	1
2025/5/15-5/17	前后端开发	解决模型处理速度慢导致前端超时的问题	周慧星	2
工作类型	工时合计 (h)	占比		
项目启动	3	1.96%		
数据收集与标注	47	30.72%		
模型训练	46	30.07%		
前后端开发	24	15.69%		
部署	24	15.69%		

阶段会议	6	3.92%		
测试	3	1.96%		
合计	153	100%		

F.会议纪要

以下为本项目开发过程中的全部会议纪要，记录了各次会议的讨论内容与决议事项。

3/12 会议

本次会议主要是了解项目情况和相关要求。

项目情况：

- 1.名称：石材幕墙污渍检测软件系统
- 2.目前已有该项目，但是识别成功率不高。
- 3.目的是把成功率从 50%提升到 70%以上。
- 4.污渍深浅问题：可以通过分级来解决

项目要求：

- 1.工作时要统计工作量，人时
- 2.每两周开一次会，得会议纪要：目标，问题，解决方案（是否解决，下一步做法）
- 3.按照现有的前端页面和后端框架来（vue+mysql）
- 4.数据集一部分会提供，另一部分自己拍

4/15 会议

会议展示：

- 1.正在标注的数据集
- 2.初步训练的模型

问题：

- 1.沟通太少，以至于很多问题无法及时解决
- 2.自己标注图片时，标注的主观性是否影响最终结果？

3.是否需负责前端页面的制作？

解决方案：

- 1.截止日期经老师提醒改到 5 月 20 日
- 2.自己标注图片的方式是对的
- 3.要分析往届的算法，找出其中的不足，以此改进自己的
- 4.前端页面直接复用已有的
- 5.学校服务器不再支持，训练需借助自己设备(笔记本电脑游戏本即可)
- 6.要多沟通，以后每周都有直面老师的沟通交流

4/22 会议

会议展示：

1.甘特图修正

对时间进度安排进行调整，加快项目进程

调整一些活动项

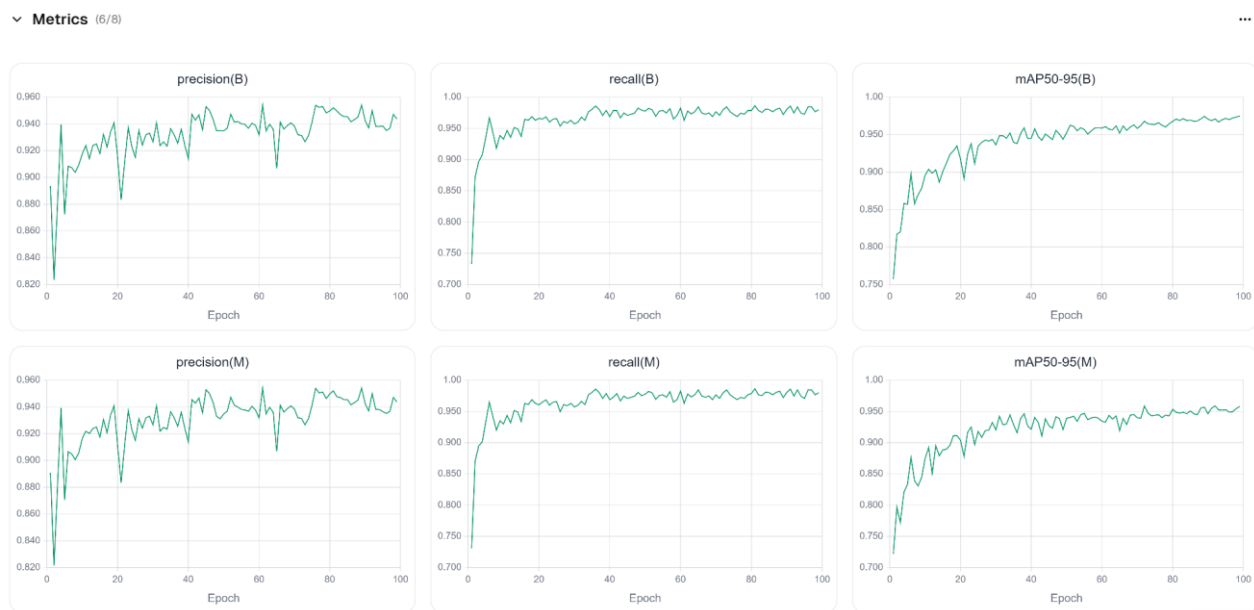


2. 原来模型分析

我们只拿到后端代码和 1 个 `best.py`（由 `yolov8n-seg` 训练而来）模型

通过分析原来的模型，它的各项指标（准确率、召回率、`map`）都很高，但实际检测效果并不好，猜测原因就是数据集质量差，数据少、标签简单

模型泛化能力弱 -> 实际检测效果差



解决方法：优化数据集质量，对污渍进行程度划分，提高标签质量，采用最新 `yolo26-seg` 模型,并进行算法改进

进度：目前数据集已收集到 720 张，包含部分原来的数据集、公开数据集平台 (`kaggle/roboflow`) 收集、自己拍摄的照片

3. 前端代码

前端代码是个总系统，而后端又是子系统，就各种 `api` 接口也接不上，数据库 `sql` 没有，压根运行不了。

自己也早就完成了前后端，并完成相应功能

把原来前端的石材污渍检测界面复制移植到了我们的前端

所以我们目的：不修改原来项目的前后端，我们的任务只要是训练出好模型，我们自己做的前后端只是方便我们展示效果（原来的实在运行不来），最后把模型交给项目组

问题:

1. 污渍目前分级是否合理?
2. 前端在直接接入困难的情况下, 上述的前端移植方案是否合理?

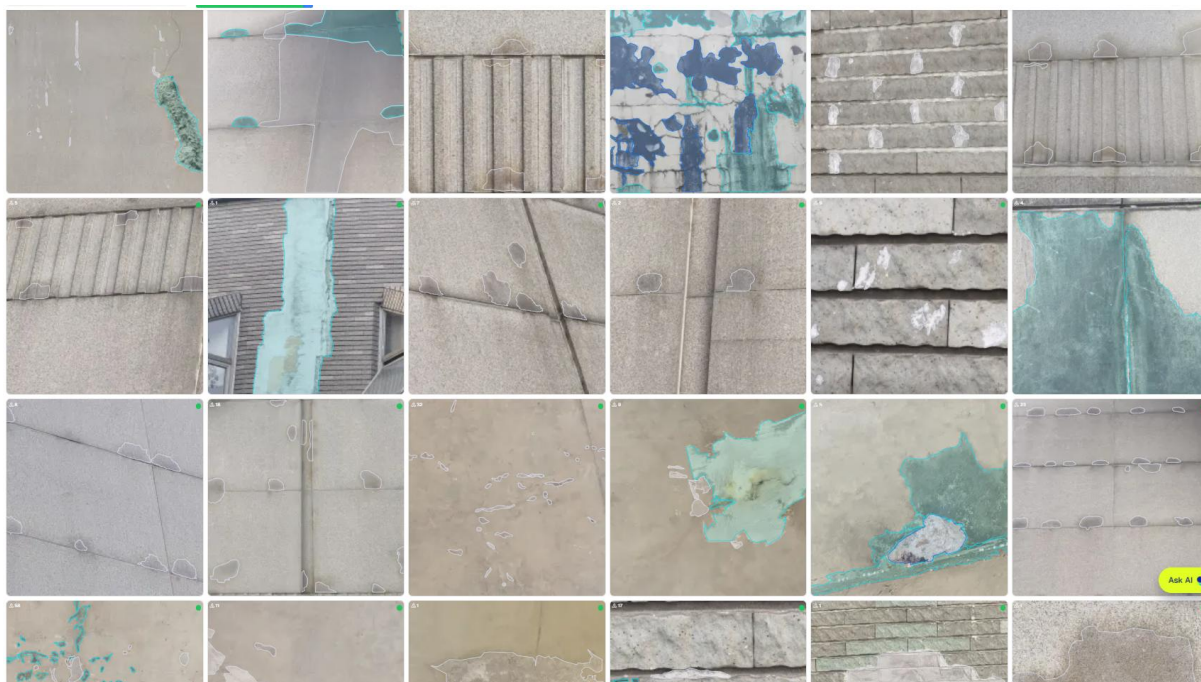
解决方案:

1. 前端不要自己做, 还是和之前的前端想办法对接
2. 在校园里可以实地拍一些污渍照片
3. 先不管前端后端, 目前主要注重于算法的选择和模型的训练
4. 污渍分三个等级就够了

4/29 会议

会议展示:

1. 在校园转了一整圈, 拍摄了一些实际的污渍图片, 并结合之前有的图片, 在“light-moderate-severe”三个等级下重新进行了标注, 目前已经标注了 300 多张图片。预期在下次会议前可将剩下的 400 张全部标注完



2. 之前已经用现有的 yolo26v 模型进行过初步训练(100 张左右小数据集), 但由于之前用的四层分级“very light-light-moderate-severe”太繁琐, 以及数据集太少, 导致效果不好

之后准备 1)在这个 yolo26v 上用新数据集再试试

2)新的算法尝试:

a.一篇论文中提到的 unet 算法

b.用新数据集在 SAM3 模型基础上训练

3.与项目前后端接通人员取得联系,获取到前端部署文档

问题:

1.目前数据集不够,划分不合理。

2.目前模型算法太单一

解决方案:

1.图片数据集由原本的七百多张经翻转等操作扩展到 3000 张,大概 70%训练集, 20%验证集, 10%测试集

2.除了 yolo26 之外, yolo11,yolo12 也可以用着试一试, 效果未必哪个更好

3.要用多种(3)算法试一试

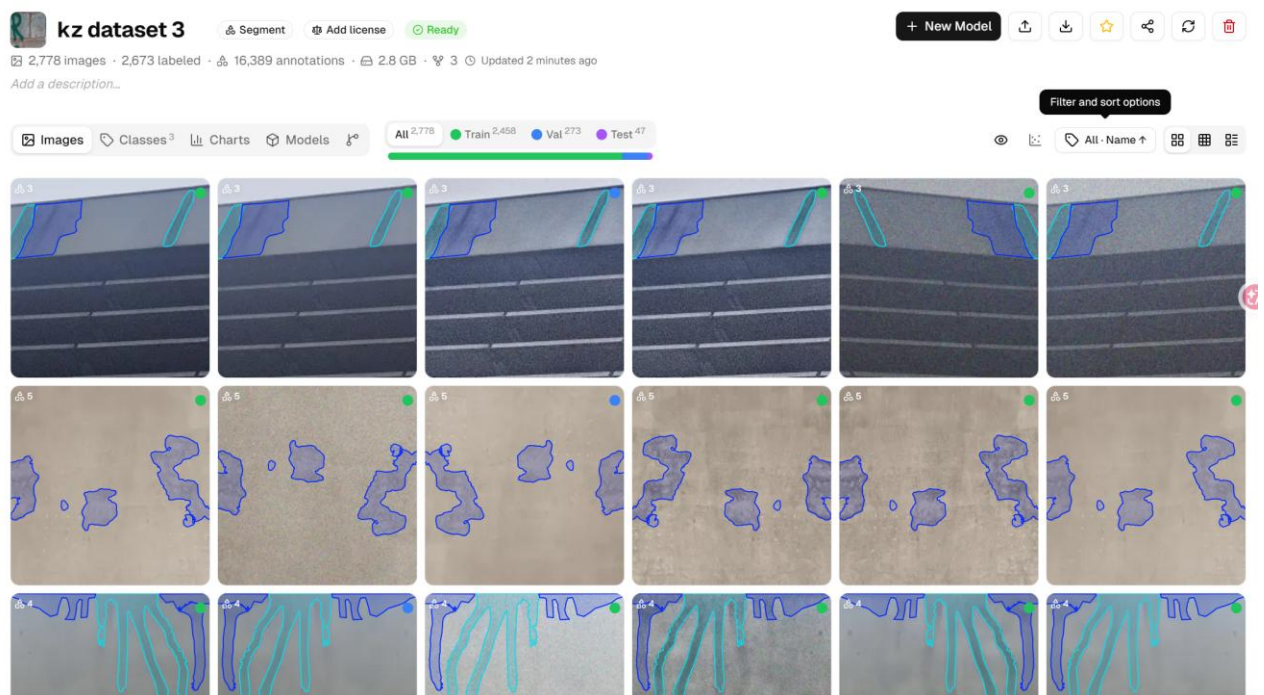
5/7 会议

会议展示:

1.数据集

经过标注、筛选最终整理出 600 多张图片

利用 python 脚本对数据进行增强(颜色增强、几何变换、局部模糊等), 最终得到 2700 多张数据



2.在 ultralytics 平台上使用 yolo 系列多个模型进行训练

训练试了 26、11、8 等版本

分 n、s、m、l、x 种，模型参数量增大，速度更慢，但精度更高

目前为了准确率，先训练了 3 个版本的 x

看起来效果不错，但只是用训练集和验证集得出的结果，真实准确率还待用之外的数据进行测试。

其他算法：之前提到的 unet 算法，训练实在太慢了，一个 epoch 就要几小时，所以暂且没用

3.下一步：

连通已有的数据库

将我们现在后端代码接入到总项目前端

问题：

1.我们的项目属于“新开发项目”还是“功能增强改进项目”？

2.后续准确率应如何来估算

3.train、val、test 三个图集的划分有问题

解决方案：

1.就用当前数据集划分 **train**、**val**、**test** 然后训练，得出的准确率即为该模型的准确率。

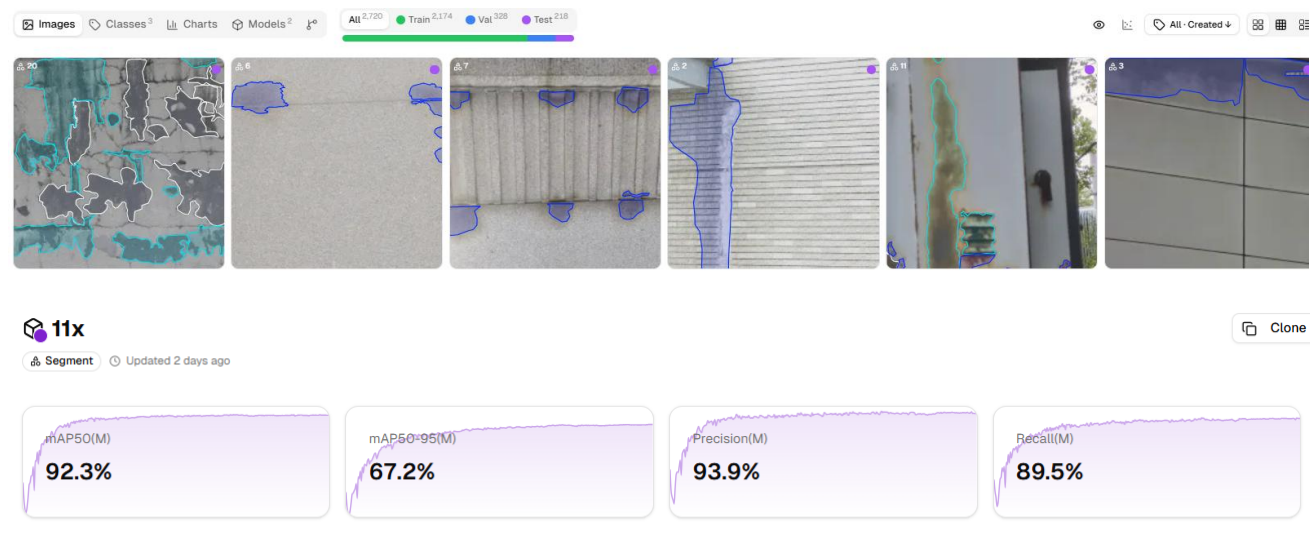
2.注意记录人时。

3.重新划分数据集，提高 **test** 测试集的占比。

5/11 会议

会议展示：

1. 添加 **test** 集后进行训练，经比较，使用 **11x** 模型，以以下比例进行训练，效果最好：



连接上数据库，将我们的后端接入原来的系统，并部署。由于 **docker** 部署失败，用了其他方式。

问题：

用其他方案部署是否可行？

展示时效果不稳定

解决方案：

1.要使用 **docker** 部署，而非用其他方式。找明之前 **docker** 部署失败的原因并修复。

2.准备好期末 **pre** 时用来展示的图片。

5/14 会议

会议展示:

- 1.已经使用 **docker** 重新部署
- 2.在前端展示了当前系统的使用效果:

上传图片

拖拽图片至此或点击上传

选择图片

支持 JPG、PNG 格式，单个文件大小不超过 50MB

检测结果

刷新 导出PDF

任务ID	67	图片名称	test01.jpg
状态	done	推理模式	本地模型
建筑	test	楼层	-
分区	x	污渍类型	moderate
污渍占比	14.49%	检测时间	2026-06-05 15:06:06
总结	检测到 6 处疑似污渍，主要类型为 moderate，污渍占比约 14.49%。		

区域明细

#	污渍类型	置信度	BBox(x1,y1,x2,y2)
1	moderate	0.9807	0.47,0.63,0.83,0.89
2	moderate	0.9713	0.00,0.74,0.16,0.94
3	severe	0.9659	0.13,0.69,0.48,0.93
4	severe	0.9648	0.75,0.60,1.00,0.88
5	moderate	0.9532	0.29,0.74,0.44,0.91
6	moderate	0.9440	0.79,0.67,0.96,0.90

检测图像

原图

检测后图片

问题:

前端展示的效果有待提高，会出现超时的情况。

解决方案:

- 1.模型处理某些图片速度太慢，以至于前端超时的问题需解决。
- 2.要准备好测试图片用于期末演示。